

AUTOMATE TASKS WITH PYTHON + AI

O'Reilly Live Training — Quick Reference Cheatsheet

Python Fundamentals · AI APIs (OpenAI · Anthropic · Ollama) · Automation Patterns

1 · PYTHON DATA TYPES & VARIABLES

Core Data Types

Type	Example	Notes
int	age = 30	Whole numbers
float	price = 9.99	Decimal numbers
str	name = "Alice"	Text, single or double quotes
bool	done = True	True / False
list	[1, 2, 3]	Ordered, mutable
dict	{"k": "v"}	Key-value pairs

Variables & F-Strings

```
name = "Ada"
age = 28
height = 1.72

# F-string formatting
print(f"Hello, {name}!")
print(f"Age: {age}, Height: {height:.1f}m")

# Type conversion
num_str = "42"
num = int(num_str)      # str → int
pi = float("3.14")      # str → float
label = str(100)        # int → str
```

String Operations

```
s = "Hello, World!"
s.upper()      # "HELLO, WORLD!"
s.lower()      # "hello, world!"
s.strip()      # remove whitespace
s.split(", ")  # ["Hello", "World!"]
s.replace("o", "0") # "Hello, W0rld!"
len(s)         # 13
s[0:5]         # "Hello" (slice)
```

2 · FUNCTIONS

Defining & Calling

```
# Basic function
def greet(name):
    return f"Hello, {name}!"

# Default parameters
def greet(name, greeting="Hi"):
    return f"{greeting}, {name}!"

# Multiple return values
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 4, 1, 5])
print(greet("Bob"))          # Hi,
                              Bob!
print(greet("Ana", "Hello")) # Hello,
                              Ana!
```

Useful Built-ins

len(x)

Length of list/string/dict

range(n)

Range of integers 0 to n-1

type(x)

Returns the type of x

input("msg")

Read user input as string

print(x, end="")

Print with custom ending

sorted(iterable)

Return new sorted list

enumerate(lst)

Index + value pairs

3 · LISTS & LOOPS

List Operations

```
fruits = ["apple", "banana", "cherry"]

# Modify
fruits.append("mango")      # add to
end
fruits.insert(1, "kiwi")    # insert
at index
fruits.remove("banana")    # remove
by value
popped = fruits.pop()      # remove
last
fruits.sort()              # sort in
place
fruits.reverse()           # reverse
in place

# Access
fruits[0]                  # first element
fruits[-1]                 # last element
fruits[1:3]                # slice
len(fruits)                # count
```

For Loops & List Comprehensions

```
# Basic for loop
for item in fruits:
    print(item)

# Loop with index
for i, item in enumerate(fruits):
    print(f"{i}: {item}")

# Loop over range
for i in range(5):         # 0,1,2,3,4
    print(i)

# List comprehension
squares = [x**2 for x in range(10)]
evens   = [x for x in range(20) if x %
2 == 0]
upper   = [s.upper() for s in fruits]
```

4 · DICTIONARIES

Dictionary Basics

```
person = {
    "name": "Alice",
    "age": 30,
    "city": "NYC"
}

# Access
person["name"]          # "Alice"
person.get("email", "N/A") # safe get
                           # with default

# Modify
person["email"] = "a@b.com" # add/
                             # update
del person["city"]          # delete
                             # key

# Methods
person.keys()             #
dict_keys(["name", "age", ...])
person.values()           #
dict_values(["Alice", 30, ...])
person.items()            # key-value pairs

# Loop
for key, val in person.items():
    print(f"{key}: {val}")
```

Nested Dictionaries

```
users = {
    "alice": {"age": 30, "role":
"admin"},
    "bob": {"age": 25, "role":
"user"},
}
users["alice"]["role"] # "admin"
users["bob"]["age"] = 26 # update
nested
```

5 · CONDITIONALS & LOGIC

If / Elif / Else

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

# One-liner (ternary)
status = "pass" if score >= 60 else
"fail"
```

Comparison & Boolean Operators

Operator	Meaning	Example
<code>==</code>	Equal	<code>x == 5</code>
<code>!=</code>	Not equal	<code>x != 0</code>
<code>></code> , <code><</code>	Greater/Less	<code>x > 10</code>
<code>>=</code> , <code><=</code>	Greater/Less or equal	<code>x <= 100</code>
<code>and</code>	Both true	<code>a and b</code>
<code>or</code>	Either true	<code>a or b</code>
<code>not</code>	Negation	<code>not done</code>
<code>in</code>	Membership	<code>"x" in lst</code>

6 · FILE I/O

Reading & Writing Files

```
# Read a text file (always use 'with')
with open("notes.txt", "r") as f:
    content = f.read()      # entire
                             file
    # lines = f.readlines() # list
                             of lines

# Write (creates or overwrites)
with open("output.txt", "w") as f:
    f.write("Hello, file!\n")

# Append to existing file
with open("log.txt", "a") as f:
    f.write("New entry\n")
```

File Modes

Mode	Meaning
"r"	Read (default, file must exist)
"w"	Write (creates new / overwrites)
"a"	Append to end of file
"r+"	Read and write

File + AI Pattern

```
# Read file, summarize with AI, write
result
with open("article.txt", "r") as f:
    text = f.read()

summary = ask_ai(f"Summarize this:
\n{text}")

with open("summary.txt", "w") as f:
    f.write(summary)
```

7 · CSV & PANDAS

Reading & Filtering CSV

```
import pandas as pd

# Load CSV
df = pd.read_csv("data.csv")

# Inspect
df.head()          # first 5 rows
df.info()          # column types
df.describe()      # stats summary

# Filter rows
high = df[df["priority"] == "High"]
cheap = df[df["price"] < 50]

# Select columns
df[["name", "price"]]

# Add a column
df["tax"] = df["price"] * 0.1
```

Creating & Exporting

```
import csv

# Write CSV with csv module
data = [{"Name", "Age"}, {"Alice", 30},
        {"Bob", 25}]
with open("people.csv", "w", newline="")
as f:
    writer = csv.writer(f)
    writer.writerows(data)

# Export DataFrame
df.to_csv("output.csv", index=False)
df.to_json("output.json",
orient="records")
```

8 · PACKAGES & APIS

Package Management

```
# Install packages
pip install requests pandas openai anthropic

# With uv (this course)
uv sync          # install from
pyproject.toml
uv run scripts/demo_script.py  # inline
PEP 723 deps auto-resolve

# In notebook
import subprocess
subprocess.run(["pip", "install", "package_name"])
```

Requests & REST APIs

```
import requests
import json
import os

API_KEY = os.getenv("MY_API_KEY") #
from .env

response = requests.get(
    "https://api.example.com/data",
    headers={"Authorization": f"Bearer {API_KEY}"},
    params={"q": "python", "limit": 10}
)

data = response.json() # parse JSON
print(response.status_code) # 200 = OK
```

Environment Variables

```
# .env file (never commit!)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...

# Load in Python
from dotenv import load_dotenv
import os

load_dotenv()
key = os.getenv("OPENAI_API_KEY")
```

9 · AI APIS — OPENAI

Chat Completions

```
from openai import OpenAI

client = OpenAI() # uses OPENAI_API_KEY
env var

response =
client.chat.completions.create(
    model="gpt-5.6-luna", # or
    "gpt-5-mini" for cheap/fast
    messages=[
        {"role": "system",
         "content": "You are a helpful
assistant."},
        {"role": "user",
         "content": "Explain recursion
simply."}
    ]
)
text =
response.choices[0].message.content
print(text)
```

Multi-Turn Conversation

```
messages = [
    {"role": "system", "content": "Be
concise."}
]

def chat(user_msg):

messages.append({"role": "user", "content"
:user_msg})
    resp =
client.chat.completions.create(
        model="gpt-5-mini",
        messages=messages)
    reply =
resp.choices[0].message.content

messages.append({"role": "assistant", "con
tent": reply})
    return reply
```

Image Generation (GPT Image 2)

```
img = client.images.generate(
    model="gpt-image-2",
    prompt="A futuristic Python logo,
    digital art",
    size="1024x1024",
    n=1
)
# Returns base64-encoded image data
image_b64 = img.data[0].b64_json
```

Audio Transcription (GPT Transcribe)

```
with open("recording.mp3", "rb") as
audio:
    transcript =
client.audio.transcriptions.create(
    model="gpt-transcribe",
    file=audio
)
print(transcript.text)
```

OpenAI Models Reference

Model	Best For
<code>gpt-5.6-luna</code>	Complex tasks, this course's default
<code>gpt-5-mini</code>	Fast, cheap, everyday use
<code>gpt-image-2</code>	Image generation
<code>gpt-transcribe</code>	Audio transcription

10 · AI APIS — ANTHROPIC (CLAUDE)

Basic Message

```
import anthropic

client = anthropic.Anthropic() # uses
ANTHROPIC_API_KEY

response = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    messages=[
        {"role": "user",
        "content": "Summarize this
article: ..."}
    ]
)
# content[0] isn't always the text block
(extended-thinking
# models can put a "thinking" block
first) – filter by type
text = next(b.text for b in
response.content if b.type == "text")
print(text)
```

With System Prompt

```
response = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=2048,
    system="You are an expert Python
tutor.",
    messages=[
        {"role": "user",
        "content": "Explain list
comprehensions."}
    ]
)
```

Claude Models

Model	Use Case
<code>claude-opus-5</code>	Most capable, complex reasoning
<code>claude-sonnet-5</code>	Balanced speed & quality, this course's default
<code>claude-haiku-4-5</code>	Fastest, lightest tasks

11 · AI APIS — OLLAMA (LOCAL)

Running Local Models

```
import ollama

# Simple text generation
response = ollama.chat(
    model="gemma4",

    messages=[{"role": "user", "content": "Hello!"}]
)
print(response.message.content)

# Using Client
client = ollama.Client()
resp = client.chat(
    model="gemma4",

    messages=[{"role": "user", "content": "Explain AI"}]
)
print(resp.message.content)
```

Structured JSON Output

```
resp = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "List 3 Python tips
as JSON"
    }],
    format="json"
)
import json
data = json.loads(resp.message.content)
```

Setup Ollama

Install from ollama.com, then: `ollama pull gemma4` to download a model. Runs 100% locally — no API key needed.

12 · UNIFIED AI INTERFACE (AI_TOOLS.PY)

The ask_ai() Function

```
# scripts/lib on sys.path (see notebooks
for the exact snippet)
from ai_tools import ask_ai,
ask_local_ai

# Cloud AI – defaults to OpenAI's
gpt-5.6-luna
result = ask_ai("Summarize this
text: ...")

result = ask_ai("Review this code: ...",
    model_name="claude-
sonnet-5")

# Local AI (Ollama) – defaults to gemma4
result = ask_local_ai("Explain
decorators")

# Structured JSON from local AI
data = ask_local_ai(
    "Return invoice data as JSON: ...",
    structured=True
)
```

parse_json_output()

```
from ai_tools import parse_json_output

raw = ask_ai(
    "Return as JSON: {name, price, qty}
for 3 items"
)
# Strips markdown code fences, parses
JSON
data = parse_json_output(raw)
print(data["items"][0]["name"])
```

13 · PROMPT ENGINEERING

Prompt Templates

```
def summarize(text, max_words=100):
    prompt = f"""
    Summarize the following text in
    {max_words} words.
    Focus on key points only.

    TEXT:
    {text}

    SUMMARY: """
    return ask_ai(prompt)

def categorize(item, categories):
    cats = ", ".join(categories)
    prompt = f"""
    Categorize this item into one of: {cats}
    Item: {item}
    Return only the category name. """
    return ask_ai(prompt)
```

Prompt Chaining

```
# Step 1: Generate idea
idea = ask_ai("Suggest a creative image
prompt")

# Step 2: Critique it
critique = ask_ai(
    f"Critique this prompt and suggest
    improvements:\n{idea}"
)

# Step 3: Refine
refined = ask_ai(
    f"Original: {idea}\nFeedback:
    {critique}\n"
    f"Write an improved version:"
)

# Step 4: Generate image
img = client.images.generate(
    model="gpt-image-2", prompt=refined)
```

Structured Output Prompt

```
prompt = """
Extract invoice data as JSON with this
schema:
{
    "vendor": "string",
    "total": number,
    "date": "YYYY-MM-DD",
    "items": [{"desc": "string", "price":
    number}]
}

Invoice text:
{invoice_text}

Return only valid JSON, no markdown."""
```

Tip: Always tell the model the exact output format you need. Say "Return only JSON, no explanation" to avoid markdown code fences you'd have to strip.

File Organizer

```
import os
import shutil
from pathlib import Path

folder = Path("./downloads")
rules = {
    ".pdf": "docs", ".jpg": "images",
    ".png": "images", ".csv": "data"
}

for file in folder.iterdir():
    if file.is_file():
        dest = rules.get(file.suffix,
            "other")
        dest_path = folder / dest
        dest_path.mkdir(exist_ok=True)
        shutil.move(str(file),
            dest_path / file.name)
```

PDF Summarizer

```
import PyPDF2

def summarize_pdf(path):
    with open(path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        text = ""
        for page in reader.pages:
            text += page.extract_text()

    return ask_ai(f"Summarize this
paper:\n{text[:4000]}")

summary = summarize_pdf("paper.pdf")
print(summary)
```

Invoice / Data Extractor

```
def extract_invoice(text):
    prompt = f"""
    Extract as JSON: vendor, total, date,
    items list.
    Text: {text}
    Return valid JSON only."""
    raw = ask_ai(prompt)
    return parse_json_output(raw)

# Batch process a folder
import os
results = []
for fname in os.listdir("invoices/"):
    with open(f"invoices/{fname}") as f:
        results.append(extract_invoice(f.read()))
)
```

Data Analysis Pipeline

```
import pandas as pd

df = pd.read_csv("sales.csv")

# Compute stats
stats = df.describe().to_string()

# Ask AI to interpret
analysis = ask_ai(
    f"Analyze these sales stats and give
    3 insights:\n{stats}"
)

# Categorize rows with AI
df["category"] =
df["description"].apply(
    lambda x: ask_ai(f"Category for:
{x}. "
                    f"Options: tech/
finance/other. "
                    f"Return one word
only.")
)
df.to_csv("sales_categorized.csv",
index=False)
```

Requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com/articles"
resp = requests.get(url)
soup = BeautifulSoup(resp.text,
"html.parser")

# Find elements
title = soup.find("h1").text
links = [a["href"] for a in
soup.find_all("a")]
articles = soup.select(".article-card")
```

Playwright (Browser Automation)

```
from playwright.sync_api import
sync_playwright

with sync_playwright() as p:
    browser =
p.chromium.launch(headless=True)
    page = browser.new_page()

    page.goto("https://example.com")
    page.fill("#search", "Python")
    page.press("#search", "Enter")
    page.wait_for_selector(".results")

    results =
page.query_selector_all(".result")
    texts = [r.inner_text() for r in
results]
    browser.close()
```

16 · QUICK REFERENCE RECIPES

Process Multiple Files with AI

```
import os

results = {}
for filename in os.listdir("docs/"):
    if filename.endswith(".txt"):
        with open(f"docs/{filename}") as
f:
            text = f.read()
            results[filename] = ask_ai(
f"One-sentence summary:
{text}"
)

# Write index
with open("index.md", "w") as f:
    for name, summary in
results.items():
        f.write(f"- **{name}**:\n
{summary}\n")
```

AI-Enhanced CSV Pipeline

```
import pandas as pd

df = pd.read_csv("tickets.csv")

# Tag priority with AI
def get_priority(desc):
    return ask_ai(
f"Rate priority (High/Med/Low):
{desc}"
).strip()

df["ai_priority"] =
df["description"].apply(get_priority)
df.to_csv("tickets_tagged.csv",
index=False)
```

Common Imports Cheatsheet

Import	Use For
<code>import os</code>	File paths, env vars, directories
<code>import json</code>	Parse / create JSON data
<code>import csv</code>	Read/write CSV files
<code>import shutil</code>	Move/copy/delete files
<code>from pathlib import Path</code>	Modern path manipulation
<code>import pandas as pd</code>	Tabular data / DataFrames
<code>import requests</code>	HTTP requests / APIs
<code>from openai import OpenAI</code>	OpenAI chat/image/audio
<code>import anthropic</code>	Anthropic Claude API
<code>import ollama</code>	Local LLMs via Ollama

RUNNING COMMANDS (UV)

`uv sync`

Install all project dependencies

`uv run scripts/script.py`

Run a script (auto-resolves its inline deps)

`uv run --with jupyter jupyter lab`

Start Jupyter Lab

`playwright install`

Install browser binaries

Course repo: Clone or open in GitHub Codespaces. Copy `.env.example` → `.env` and add your API keys before running any AI notebooks.